Technische Dokumentation

„the nailshop“

Projekt: Einstieg in die Webprogrammierung

Dokumentation der Anwendung, Architektur, Installation und Tests

Inhaltsübersicht

1. PROJEKTÜBERSICHT 3

2. VERWENDETE TECHNOLOGIEN 3

3. SOFTWAREARCHITEKTUR 4

4. DATENBANKSTRUKTUR 5

5. IMPLEMENTIERTE FUNKTIONEN 6

6. SICHERHEIT 8

7. INSTALLATION UND AUSFÜHRUNG 9

Voraussetzungen 9

Anwendung starten 9

Stripe prüfen 10

Stoppen und Fehler prüfen 10

8. TESTDURCHFÜHRUNG 11

9. BEKANNTE EINSCHRÄNKUNGEN 12

10. LITERATURVERZEICHNIS 13

1. Projektübersicht

Ziel des Projekts war eine datenbankgestützte Webanwendung, in der Frontend, Backend und Datenbank zusammenarbeiten. Neben den Shop-Funktionen sollten auch Benutzerverwaltung, Zugriffskontrolle und die Zahlungsanbindung über Stripe umgesetzt werden.

Der Nailshop ist ein Online-Shop für Nagellacke. Kunden können Produkte ansehen, in den Warenkorb legen und bestellen. Eine Bestellung ist mit oder ohne Kundenkonto möglich. Mit einem Kundenkonto können Kunden ihre Adressen verwalten und ihre Bestellungen ansehen. Admins verwalten Produkte und Kategorien. Die Bezahlung erfolgt über Stripe im Testmodus. Dabei werden keine echten Zahlungen durchgeführt (vgl. Stripe: Testing).

2. Verwendete Technologien

Technologie	Verwendung im Projekt
Java 17	Enthält die Anwendungslogik.
Spring Boot	Bildet die Grundlage der Webanwendung (vgl. Spring Boot: Übersicht).
Thymeleaf	Fügt Daten aus dem Backend in HTML-Vorlagen ein (vgl. Thymeleaf).
HTML	Definiert die Seiten und Formulare (vgl. WHATWG: HTML Forms).
CSS	Gestaltet die Seiten. Media Queries passen das Layout an verschiedene Bildschirmgrößen an (vgl. W3C: Media Queries).
JavaScript	Steuert das Karussell (Homepage), Bestätigungsfenster und das Hinzufügen zum Warenkorb ohne Seitenwechsel.
MySQL	Speichert Produkte, Kategorien, Kunden, Adressen und Bestellungen.
Spring Data JPA	Verbindet die Entity-Klassen und Repositories mit der Datenbank (vgl. Spring Data JPA).
Spring Security	Regelt den Login und die Zugriffsrechte (vgl. Spring Security: UserDetailsService und Authorization).
Stripe	Verarbeitet Zahlungen im Testmodus (vgl. Stripe: Testing).
Maven	Verwaltet Dependencies und erstellt und startet die Anwendung (vgl. Maven).

Technologie	Verwendung im Projekt
Docker und Docker Compose	Starten die Anwendung, MySQL und die Stripe CLI in Containern (vgl. Docker Compose).
Colima	Stellt auf dem Mac die Umgebung für Docker bereit (vgl. Colima).

Ich habe Spring Boot und Thymeleaf gewählt, weil ich die HTML-Seiten direkt mit Daten aus dem Backend erstellen kann (vgl. Thymeleaf: Verwendung mit Spring). Für diesen Aufbau brauche ich kein separates Frontend-Framework. Ich habe MySQL gewählt, weil Produkte, Kunden und Bestellungen dauerhaft gespeichert werden sollen. Im Docker-Aufbau bleiben die Daten über ein Volume erhalten (vgl. MySQL: Datenhaltung im Container).

3. Softwarearchitektur

Ich habe die Anwendung in Controller, Services, Repositories und Entities aufgeteilt, um die Aufgaben klar zu trennen. So ist es bei Änderungen leichter zu erkennen, welcher Teil der Anwendung betroffen ist. Komplexere Anwendungslogik liegt in den Services. Die Repositories übernehmen den Datenbankzugriff. Spring stellt die benötigten Abhängigkeiten über die Konstruktoren bereit. Dies entspricht Constructor Dependency Injection (vgl. Spring Framework).

Bestandteil	Aufgabe
Controller	Empfangen HTTP Requests. Sie verarbeiten Formulardaten und geben eine View oder einen Redirect zurück.
Services	Enthalten Anwendungslogik, zum Beispiel für den Warenkorb, Bestellungen und Stripe-Zahlungen.
Repositories	Lesen und speichern Daten über Spring Data JPA (vgl. Spring Data JPA: Repositories).
Entities	Beschreiben gespeicherte Daten und ihre Beziehungen. JPA ordnet sie den Datenbanktabellen zu.
Thymeleaf Views	Erstellen HTML-Seiten mit den Daten, die der Controller im Model bereitstellt.

Die Aufgaben der Controller und das Übergeben eines Models an eine View werden in der Spring MVC Dokumentation beschrieben. Für die HTML-Darstellung, Thymeleaf hilft.

Beim Checkout sendet der Browser die Formulardaten an den CheckoutController. Dieser prüft die Kontaktdaten und Lieferadresse. Danach ruft er den OrderService auf. Der Service lädt die Produkte aus der Datenbank und prüft Mengen, Preise und Lagerbestand. Anschließend speichert er die Bestellung und ihre OrderItems über die Repositories. Dies erfolgt innerhalb einer

Transaction mit @Transactional (vgl. Spring Framework). Der Controller leert danach den Warenkorb und leitet zur Bestellseite weiter. Die Zahlung erfolgt in einem separaten Schritt.

Der HomeController lädt zum Beispiel Kategorien und Produkte und übergibt sie an die Thymeleaf View der Startseite. Für diesen Ablauf habe ich keinen zusätzlichen Service eingeführt. Die Bestellabwicklung habe ich dagegen in den OrderService ausgelagert. Dort kommen mehrere Prüfungen und Datenbankoperationen zusammen.

4. Datenbankstruktur

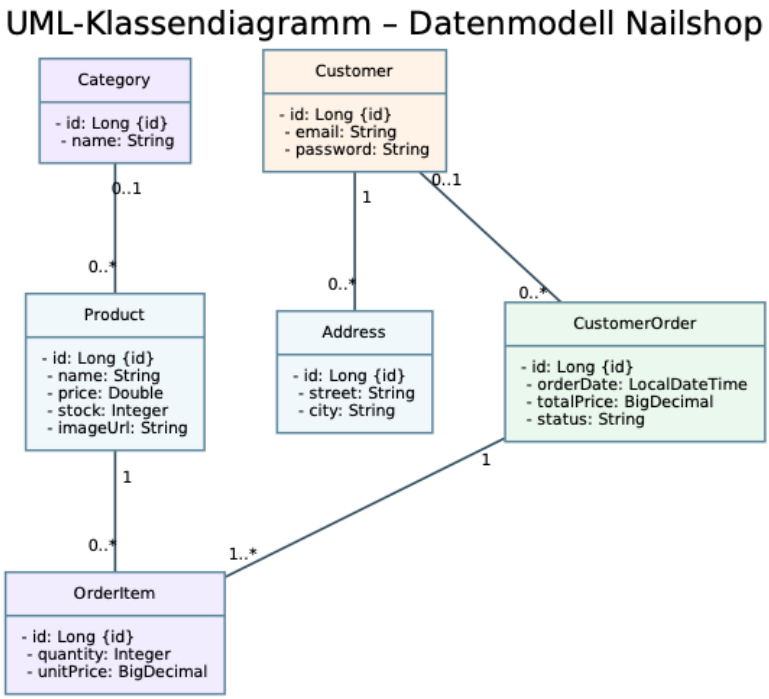
Die Datenbank enthält sechs Entities. Jede Entity besitzt eine automatisch erzeugte ID als Primary Key. Die automatische Erzeugung wird mit @GeneratedValue festgelegt. Beziehungen werden über Foreign Keys abgebildet. Die Java-Klassen verwenden dafür die Annotation @ManyToOne. Diese bildet eine Beziehung von mehreren Datensätzen zu einem Entity ab.

Entity	Gespeicherte Daten
Product	Name, Beschreibung, Preis, Lagerbestand, Bildpfad und Kategorie.
Category	Name und Rabatt in Prozent.
Customer	Vorname, Nachname, eindeutige E-Mail-Adresse, Passwort-Hash und Rolle.
Address	Straße, Hausnummer, Postleitzahl, Stadt, Land und zugehöriger Customer FK.
CustomerOrder	Bestelldatum, Gesamtpreis, Status, Kontaktdaten, Lieferadresse und Stripe Session ID. Bei Kundenbestellungen wird auch der Customer gespeichert.
OrderItem	Zugehörige Bestellung, Product, Menge und Einzelpreis zum Bestellzeitpunkt.

Eine Category kann mehreren Products zugeordnet sein. Ein Product hat höchstens eine Category. Ein Product kann auch ohne Category gespeichert werden. Der Rabatt der Category wird bei der Preisberechnung berücksichtigt.

Ein Customer kann mehrere Addresses und CustomerOrders haben. Jede gespeicherte Kundenadresse wird in der Anwendung einem Customer zugeordnet. Die Bestellung kann dagegen ohne Customer angelegt werden. Dadurch ist ein Gastkauf möglich. Kontaktdaten und Lieferadresse stehen direkt in der CustomerOrder. Es gibt keine direkte Beziehung zwischen CustomerOrder und Address. Ich kopiere die Lieferdaten einer gespeicherten Adresse in die Bestellung, damit die ursprüngliche Lieferadresse erhalten bleibt. Auch wenn ein Kunde die gespeicherte Adresse später bearbeitet oder löscht, bleiben die Lieferdaten der Bestellung unverändert.

Eine CustomerOrder enthält eine oder mehrere OrderItems. Der OrderService verhindert Bestellungen mit einem leeren Warenkorb. Jedes OrderItem verweist auf genau eine CustomerOrder und ein Product. Ein Product kann in vielen OrderItems vorkommen. OrderItem verbindet damit Bestellungen und Produkte. Es speichert zusätzlich die Menge und den Einzelpreis. Den Preis zum Bestellzeitpunkt wird gespeichert im OrderItem, damit der



ursprüngliche Kaufpreis nachvollziehbar bleibt. Spätere Änderungen am Produktpreis oder Kategorienrabatt verändern diesen Preis nicht mehr.

CartItem ist kein Entity. Die Warenkorbdaten werden im sessionbezogenen CartService gehalten und nicht in einer Datenbanktabelle gespeichert.

Das UML-Klassendiagramm zeigt die sechs Entities und ihre Beziehungen als vereinfachte Übersicht.

5. Implementierte Funktionen

Produkte und Kategorien

Der ProductController lädt Produkte aus der Datenbank. Über eine Category ID können Kunden die Liste filtern. Admins können Produkte anlegen, bearbeiten und löschen. Beim Speichern prüft der Controller den Namen, Preis und Lagerbestand. Produktbilder werden als Dateien gespeichert. Die Datenbank enthält den Bildpfad.

Admins können auch Kategorien verwalten und einen Rabatt zwischen 0 - 100 Prozent festlegen. Dieser Rabatt gilt für die zugeordneten Produkte. Eine Kategorie kann erst gelöscht werden, wenn ihr keine Produkte mehr zugeordnet sind.

Registrierung und Login

Bei der Registrierung prüft der RegistrationController das E-Mail-Format und ob bereits ein Konto mit dieser Adresse existiert. Das Passwort wird mit BCrypt gehasht (vgl. Spring Security: Password Storage). Neue Konten erhalten nur die Rolle CUSTOMER. Nach der Registrierung wird der Kunde automatisch eingeloggt. Beim späteren Login lädt der CustomerUserDetailsService das Konto anhand der E-Mail-Adresse. Spring Security prüft das Passwort und leitet zum Kundenkonto weiter.

UserDetailsService stellt Spring Security die Kontodaten für die Authentifizierung bereit (vgl. Spring Security: UserDetailsService).

Warenkorb

Der CartService verwaltet einen eigenen Warenkorb pro HTTP-Session. Er verwendet dafür @SessionScope (vgl. Spring Framework: Bean Scopes). Für den Warenkorb würde die Session benutzt, damit auch Gäste ohne Kundenkonto Produkte hinzufügen können. Mit dem Ende der Session geht allerdings auch der Warenkorb verloren. Kunden können Produkte hinzufügen, Mengen ändern und Artikel entfernen. Beim Hinzufügen und Ändern wird der verfügbare Lagerbestand geprüft. Bei unzureichendem Bestand wird die Änderung abgelehnt. Die Preise kommen aus den Produktdaten. Der Kategorienrabatt wird bei der Berechnung berücksichtigt. Dazu habe ich JavaScript benutzt, denn das ermöglicht das Hinzufügen ohne Seitenwechsel.

Check-out und Bestellungen

Der Check-out ist mit oder ohne Kundenkonto möglich. Kunden geben ihre Kontaktdaten und Lieferadresse ein. Eingeloggte Kunden können eine eigene gespeicherte Adresse auswählen. Der Controller prüft die Eingaben und die Zuordnung der Adresse.

Vor dem Speichern lädt der OrderService die Produkte erneut aus der Datenbank. Er prüft Mengen, Preise und Lagerbestand und berechnet den Gesamtpreis. Bei einem Fehler wird keine Bestellung gespeichert. Bei Erfolg entstehen eine CustomerOrder und die zugehörigen OrderItems. Die Bestellung erhält den Status PENDING_PAYMENT. Danach wird der Warenkorb geleert. Den Lagerbestand wird reduziert erst nach einer bestätigten Zahlung. Dadurch verändern abgebrochene Zahlungen den verfügbaren Bestand nicht.

Im Kundenkonto können Kunden Adressen anlegen, bearbeiten und löschen. Ihre Bestellungen werden mit den jeweiligen OrderItems angezeigt. Vor dem Anzeigen oder Bezahlen einer Bestellung wird der Zugriff geprüft.

Stripe-Zahlungen

Die Zahlung wird auf der Bestellseite gestartet. Der StripePaymentService erstellt eine Stripe Checkout Session mit dem Gesamtbetrag in Euro. Der Browser wird anschließend zur Stripe-Seite weitergeleitet. Stripe Checkout bietet dafür eine gehostete Zahlungsseite (vgl. Stripe: Checkout).

Stripe meldet das Zahlungsergebnis über einen Webhook (vgl. Stripe: Webhooks). Der Controller prüft die Signatur und lädt den aktuellen Zahlungsstatus von Stripe. Der OrderService prüft den Betrag, die Währung und erneut den Lagerbestand. Bei erfolgreicher Verarbeitung wird der Bestand reduziert und der Status auf PAID gesetzt. Bereits verarbeitete Zahlungen werden nicht erneut verbucht. Vor der Zahlung werden keine Produkte reserviert. In der Zwischenzeit können andere Bestellungen den Bestand reduzieren. Ein Abbruch auf der Stripe-Seite lässt die Bestellung offen. Sie kann später erneut bezahlt werden.

6. Sicherheit

Login und Zugriffsrechte

Für den Login und die Zugriffsrechte habe ich Spring Security benutzt. Die Authentication prüft die Identität beim Login: Die Authorization legt fest, auf welche Funktionen und Daten das Konto zugreifen darf (vgl. Spring Security: Authentication und Authorization). Zugriffsregeln können auch an Rollen gebunden werden (vgl. Spring Security: Authorization). Im Projekt dürfen nur Konten mit der Rolle ADMIN auf `/admin/**` zugreifen. Der Kundenbereich ist für die Rollen CUSTOMER und ADMIN freigegeben. Neue Konten erhalten bei der Registrierung immer die Rolle CUSTOMER.

Passwörter

Passwörter werden mit dem BCryptPasswordEncoder als Hash gespeichert. BCrypt ist eine adaptive Einwegfunktion: beim Login wird das eingegebene Passwort gegen den gespeicherten Hash geprüft (vgl. Spring Security: Password Storage). Also, die Anwendung speichert das Passwort bei der Registrierung nicht im Klartext.

Zugriff auf Kundendaten

Zusätzlich zu den Rollen werden in den Controllern Prüfungen durchgeführt, wem eine Adresse oder Bestellung gehört. Die Rolle CUSTOMER allein erlaubt keinen Zugriff auf alle Kundendaten. Durch die zusätzliche Prüfung kann ein Kunde nicht über eine fremde ID auf diese Daten zugreifen.

CSRF und Stripe Webhook

Spring Security schützt standardmäßig Requests mit Methoden wie POST durch CSRF-Tokens. Damit werden unerwünschte Requests über fremde Webseiten erschwert (vgl. Spring Security: CSRF). Im Projekt ist nur `/stripe/webhook` von dieser Prüfung ausgenommen.

Der StripeWebhookController prüft stattdessen die Stripe-Signatur. Dafür verwendet er den unveränderten Request Body, den Header `Stripe-Signature` und das Webhook Secret. Stripe beschreibt diese Prüfung zur Kontrolle der Herkunft des Events (vgl. Stripe: Webhook Signatures). Danach lädt die Anwendung die Checkout Session von Stripe und prüft den Zahlungsstatus, Betrag und die Währung vor der Verarbeitung.

Eingabeprüfung

Die Anwendung prüft wichtige Eingaben auch im Backend. Dazu gehören Produktname, Preis, Lagerbestand, Kategorienrabatt und Lieferdaten. Beim Checkout werden die Preise aus der Datenbank verwendet. Die Registrierung prüft das E-Mail-Format und bereits vorhandene Konten.

Bekannte Schwächen

Das Löschen von Produkten verwendet aktuell GET. Dabei werden Daten verändert, ohne dass die normale CSRF-Token-Prüfung greift.

Auch bei Kategorienamen und Bild-Uploads bestehen noch Prüfungslücken. Die vorhandenen Schutzmechanismen lösen diese Probleme nicht automatisch. Abschnitt 8 beschreibt die dokumentierten Testergebnisse.

7. Installation und Ausführung

Voraussetzungen

Auf dem Host müssen Docker und Docker Compose installiert und gestartet sein. Auf dem Mac kann dafür Colima verwendet werden. Java 17, Maven, MySQL und die Stripe CLI werden über die Container bereitgestellt für eine einfaches Setup, deswegen eine zusätzliche lokale Installation dieser Programme ist für diesen Setup nicht nötig. Für den ersten Build und Stripe-Zahlungen wird eine Internetverbindung benötigt. Port 8080 muss frei sein. Zusätzlich: Wenn MacOS verwendet wird, muss geachtet werden, dass der Docker Container in der Colima VM eingerichtet ist, nicht lokal.

Anwendung starten

Docker Compose verwaltet drei Dienste: `app` für die Spring-Boot-Anwendung, `db` für MySQL und `stripe` für die Weiterleitung der Webhook Events bei Testzahlungen.

Alle folgenden Docker-Befehle müssen auf dem Host ausgeführt werden:

```
docker compose -f compose.yaml up -d --build
```

`-f compose.yaml` wählt die gemeinsame Konfiguration aus. `--build` erstellt die Images. `-d` startet die Container im Hintergrund (vgl. Docker: compose up). Die Dienste `app`, `db` und `stripe` werden gestartet. Der App-Container führt automatisch `mvn spring-boot:run` aus. Dieser Maven-Befehl startet

die Spring-Boot-Anwendung (vgl. Spring Boot: Maven Run). Er muss bei diesem Startweg nicht zusätzlich eingegeben werden. (Vorgang mit neuem Container getestet)

Beim ersten Start einer neuen Datenbank wird das SQL-Skript aus docker/mysql/init/ geladen. Es enthält Produkte, Kategorien, Demo-Konten und Beispielbestellungen, für bessere Testierung. Das MySQL-Image führt Initialisierungsskripte aus /docker-entrypoint-initdb.d/ beim Anlegen der Datenbank aus (vgl. MySQL: Initialisierung im Container). Die Daten bleiben im Volume mysql-data erhalten.

Nach der Meldung Started NailshopDemoApplication im Browser http://localhost:8080 öffnen. Der erste Start kann wegen Downloads länger dauern.

Demo-Konto	E-Mail	Passwort
Admin	admin@thenailshop.com	admin
Customer	customer@thenailshop.com	customer

Stripe prüfen

Die Compose-Datei enthält die Testkonfiguration für Stripe. Für Zahlungen muss der Test API Key gültig sein. Das Webhook Secret der Anwendung muss zum laufenden Stripe Listener passen. Die Ausgabe kann auf dem Host geprüft werden:

```
docker compose -f compose.yaml logs --tail=50 stripe
```

Falls das angezeigte Secret von STRIPE_WEBHOOK_SECRET in compose.yaml abweicht, den Wert dort anpassen

Stoppen und Fehler prüfen

Zum Stoppen auf dem Host ausführen:

```
docker compose down
```

Der Befehl entfernt Container und das Compose-Netzwerk. Die benannten Volumes bleiben erhalten (vgl. Docker: compose down).

Bei einem Datenbankfehler die Logs mit docker compose -f compose.yaml logs --tail=50 db prüfen. Ist Port 8080 belegt, die andere Anwendung auf diesem Port beenden. Bei Problemen mit der Portweiterleitung auf dem Mac kann Colima auf dem Host mit colima stop und danach colima start --port-forwarder=ssh neu gestartet werden. Die Option wählt den SSH Port Forwarder aus (vgl. Colima: Start-Optionen). Anschließend den Docker-Startbefehl erneut ausführen.

8. Testdurchführung

Ich habe die Anwendung in einem neuen Docker-Container getestet. Der Start und das Laden der Demodaten aus docker/mysql/init/ waren erfolgreich. Auch die Anwendung und die Stripe-Testzahlung habe ich im Browser auf meinem Mac erfolgreich getestet.

Tests wurden während der Entwicklung durchgeführt. Die folgende Tabelle fasst die dokumentierten Ergebnisse zusammen.

Test	Ergebnis
Seitenaufruf	Startseite, Produktliste, Warenkorb, Login und Registrierung waren erreichbar.
Registrierung und Login	Ein Testkonto konnte angelegt werden. Beide Demo-Konten konnten sich einloggen.
Zugriffsrechte	Gäste wurden zum Login weitergeleitet. Ein Customer konnte keine Admin-Seiten öffnen. Fremde Bestellungen und Adressen waren nicht zugänglich.
Adressen	Eine Adresse konnte gespeichert und beim Checkout ausgewählt werden.
Warenkorb	Mengen konnten erhöht und reduziert werden. Die Gesamtsumme wurde angepasst. Ein Produkt ohne Bestand konnte nicht hinzugefügt werden.
Preisberechnung	Ein mitgesendeter falscher Preis wurde ignoriert. Die Anwendung verwendete den Preis aus der Datenbank mit Kategorienrabatt.
Checkout	Der Gast-Checkout war erreichbar. Beim Checkout mit Kundenkonto wurde eine Bestellung mit PENDING_PAYMENT gespeichert.
Stripe-Testzahlung	Die Zahlung wurde mehrmals bestätigt. Der Webhook setzte die Bestellung auf PAID. Der Lagerbestand sank um die bestellte Menge. Ein erneuter Seitenaufruf reduzierte ihn nicht nochmals.
Bestellübersicht	Die bezahlte Bestellung wurde angezeigt und blieb nach erneutem Login sichtbar.
Schutzmechanismen	POST Requests ohne CSRF-Token und ein Webhook mit ungültiger Signatur wurden abgelehnt.
Produktformular	Leere Pflichtangaben wurden abgelehnt. Es wurde kein Produkt gespeichert.

Die Tests zeigten auch leider manche Fehler, die bearbeitet würden, wie z.B. Produkte konnten ohne erforderliche Attribute angelegt werden.

Nach aktuellen Status: Produkte können gespeichert werden nur wenn Name, Menge und Preis vorhanden sind.

9. Bekannte Einschränkungen

Die Anwendung ist nur auf Deutsch verfügbar. Eine DE/EN-Umschaltung ist nicht mehr implementiert. Kunden ohne Deutschkenntnisse können den Shop dadurch nur eingeschränkt nutzen.

Stripe ist im Testmodus eingebunden. Echte Zahlungen sind nicht Teil des Projekts (vgl. Stripe: Testing). Für Testzahlungen werden ein gültiger Test API Key und ein zum Stripe Listener passendes Webhook Secret benötigt (vgl. Stripe: Webhook-Konfiguration).

Der Warenkorb wird nur für die aktuelle HTTP-Session gespeichert. Der verwendete Session Scope bindet den CartService an diese Session (vgl. Spring Framework: Bean Scopes). Er wird nicht dauerhaft mit dem Kundenkonto verbunden. Geht die Session verloren, müssen Kunden die Produkte erneut hinzufügen. Auch der Zugriff auf eine Gastbestellung hängt von der Session ab. Nach dem Verlust der Session ist diese Bestellung über die bisherige Bestellseite nicht mehr zugänglich.

Vor der Zahlung werden Produkte nicht reserviert. Eine erfolgreiche Zahlung garantiert deshalb nicht, dass der Bestand noch ausreicht. Der Status PAID_STOCK_ISSUE kennzeichnet diesen Fall. Eine automatische Lösung ist nicht implementiert. Die weitere Bearbeitung muss manuell geklärt werden.

Für Produkte ohne Bild gibt es noch kein Ersatzbild. Dadurch kann in der Produktansicht ein fehlendes Bild angezeigt werden.

10. Literaturverzeichnis

Colima Project (o. J.). Colima – Container runtimes on macOS and Linux. Verfügbar unter: https://colima.run/

Colima Project (o. J.). Colima – Start command (Quellcode). GitHub. Verfügbar unter: https://github.com/abiosoft/colima/blob/main/cmd/start.go

Docker Inc. (o. J.). How Compose works. Verfügbar unter: https://docs.docker.com/compose/intro/compose-application-model/

Docker Inc. (o. J.). docker compose down. Verfügbar unter: https://docs.docker.com/reference/cli/docker/compose/down/

Docker Inc. (o. J.). docker compose up. Verfügbar unter: https://docs.docker.com/reference/cli/docker/compose/up/

Apache Software Foundation (o. J.). Introduction to the Dependency Mechanism – Maven. Verfügbar unter: https://maven.apache.org/guides/introduction/introduction-to-dependency-mechanism.html

Oracle (o. J.). MySQL 8.0 Reference Manual – More Topics on Deploying MySQL Server with Docker. Verfügbar unter: https://dev.mysql.com/doc/refman/8.0/en/docker-mysql-more-topics.html

Spring (o. J.). Spring Boot Reference Documentation. Verfügbar unter: https://docs.spring.io/spring-boot/index.html

Spring (o. J.). Spring Boot – Running Your Application with Maven. Verfügbar unter: https://docs.spring.io/spring-boot/maven-plugin/run.html

Spring (o. J.). Spring Data JPA – Reference Documentation. Verfügbar unter: https://docs.spring.io/spring-data/jpa/reference/jpa.html

Spring (o. J.). Spring Framework – Dependency Injection. Verfügbar unter: https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html

Spring (o. J.). Spring Framework – Bean Scopes. Verfügbar unter: https://docs.spring.io/spring-framework/reference/core/beans/factory-scopes.html

Spring (o. J.). Spring Framework – Using @Transactional. Verfügbar unter: https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/annotations.html

Spring (o. J.). Spring Framework – Annotated Controllers. Verfügbar unter: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller.html

Spring (o. J.). Spring Security – Authentication. Verfügbar unter: https://docs.spring.io/spring-security/reference/servlet/authentication/index.html

Spring (o. J.). Spring Security – Authorization. Verfügbar unter: https://docs.spring.io/spring-security/reference/servlet/authorization/index.html

Spring (o. J.). Spring Security – Authorize HTTP Requests. Verfügbar unter: https://docs.spring.io/spring-security/reference/servlet/authorization/authorize-http-requests.html

Spring (o. J.). Spring Security – Cross Site Request Forgery (CSRF). Verfügbar unter: https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html

Spring (o. J.). Spring Security – Password Storage. Verfügbar unter: https://docs.spring.io/spring-security/reference/7.0/features/authentication/password-storage.html

Spring (o. J.). Spring Security – UserDetailsService. Verfügbar unter: https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/user-details-service.html

Stripe (o. J.). Accept a Payment with Checkout. Verfügbar unter: https://docs.stripe.com/payments/checkout

Stripe (o. J.). Manage Webhook Endpoints. Verfügbar unter: https://docs.stripe.com/events/manage-webhook-endpoints

Stripe (o. J.). Resolve Signature Verification Errors. Verfügbar unter: https://docs.stripe.com/events/manage-webhook-endpoints#signature-errors

Stripe (o. J.). Testing. Verfügbar unter: https://docs.stripe.com/testing

Thymeleaf (o. J.). Tutorial: Using Thymeleaf (Version 3.1). Verfügbar unter: https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html

WHATWG (o. J.). HTML Living Standard – Forms. Verfügbar unter: https://html.spec.whatwg.org/multipage/forms.html

World Wide Web Consortium (W3C) (2012). Media Queries Level 3. Verfügbar unter: https://www.w3.org/TR/mediaqueries-3/